Complete step-by-step manual

BugIt

QA Bug-Filing Agent for VS Code

First-Time Setup & Everyday Use — a detailed, click-by-click guide written for people who have never done anything like this before. No technical background needed. Follow it in order and you'll be filing clean bug tickets today.

Runs in: VS Code — Windows (primary), macOS & Linux

WHAT'S INSIDE

Do **Part 1** once, in order. Everything after that you can dip into whenever you need it. The **Troubleshooting** and **FAQ** near the end answer the most common questions — please check them before emailing support.

> **The one rule that keeps you safe**
>
> BugIt **never** files anything without showing you a preview and getting your typed "yes" first. You are always in control. Want to practise with zero risk? Type `dry run` and it writes the whole report **without** filing.

> **First-time setup vs. everyday use**
>
> You only follow all of **Part 1** once. After that, opening BugIt is quick — and updates are one command (`Update`). See Part 4.

PART 1 · SETUP — DO THIS ONCE

Roughly 15 minutes. Do the steps in order — each one builds on the last. Don't skip ahead.

0 Get GitHub Copilot

> **You cannot use BugIt without this**
>
> BugIt is the "driver" — GitHub Copilot is the "engine" (the AI that actually writes the reports). If you don't have Copilot working in VS Code, nothing below will help. Set this up first.

What it is: GitHub Copilot is an AI assistant from GitHub (owned by Microsoft). It has a free tier for light use and a paid tier (about $10/month) for heavy use. You'll install it in Step 1 and sign in.

1. You'll need a **GitHub account**. If you don't have one, create a free one at `github.com/signup` .
2. Copilot's free tier is enough to try BugIt. If you file bugs all day, the paid "Copilot Pro" plan is worth it — you can upgrade anytime at `github.com/features/copilot` .
3. You'll actually turn Copilot on inside VS Code in Step 1 — no need to do anything else here yet.

1 Install VS Code

VS Code is a free Microsoft app. It's the "window" BugIt lives in.

1. Open your web browser and go to `code.visualstudio.com`.
2. Click the big blue **Download** button (it detects your system automatically).
3. Open the downloaded file and install it — **accept all the default options**, just keep clicking Next/Install.
4. Open VS Code once it's installed.

Turn on GitHub Copilot inside VS Code

1. On the far left of VS Code, click the **Extensions** icon (it looks like four small squares).
2. In the search box, type **GitHub Copilot**.
3. Click **Install** on "GitHub Copilot" (and, if offered, "GitHub Copilot Chat").
4. A prompt will ask you to **Sign in to GitHub** — click it and sign in with your GitHub account in the browser window that opens.

> ✅ **You'll know it worked when:** a small Copilot icon appears at the bottom of VS Code, and the Chat panel opens (the speech-bubble icon on the left, or press `Ctrl+Alt+I`).

2 Unzip your BugIt download

Your purchase came with a .zip file (for example `bugit-qa-agent.zip`). You need to unzip it — running it zipped won't work.

1. Find the .zip file (usually in your **Downloads** folder).
2. **Right-click** it → choose **Extract All...** (Windows) or double-click it (Mac).
3. Choose a simple location like your **Documents** folder, then click Extract.
4. You'll now have a **BugIt** folder with lots of files inside. Remember where it is.

> **Don't put it in OneDrive or a synced folder**
>
> Cloud-sync folders can cause file conflicts. Your Documents or Desktop is perfect. Also don't rename the files inside — just leave the folder as-is.

3 Open BugIt in VS Code

1. In VS Code, click **File** → **Open Folder** (top-left menu).
2. Navigate to where you unzipped BugIt (e.g. Documents).
3. Click the **BugIt** folder once to select it, then click **Select Folder**.
4. If VS Code asks "Do you trust the authors of the files in this folder?", click **Yes, I trust the authors**.

> ✅ **You'll know it worked when:** you see the BugIt files (folders like `tools`, `.github`, and files like `README.md`) listed down the left side of VS Code.

4 Pick the BugIt agent in chat

1. Open the Copilot **Chat** panel — click the speech-bubble icon on the left, or press `Ctrl+Alt+I` (Windows) / `Cmd+Alt+I` (Mac).
2. At the top of the chat box there's a small **mode dropdown** (it may say "Ask" or "Agent").
3. Click it and choose **BugIt QA Agent** from the list.

> **Don't see "BugIt QA Agent" in the list?**
> Make sure you opened the BugIt folder (Step 3), not a different folder — the agent loads from inside it. Close and reopen the folder if needed.

5 Activate your license

A BugIt account is all you need — there is no license key to copy or paste. Activation happens in your browser.

1. In the chat box, type `hi` and press Enter.
2. Type `Activate`. BugIt opens the **BugIt Portal** in your browser. Sign in to your own account, choose your Solo or Team entitlement, and approve this device. Then return to VS Code — BugIt finishes authorizing automatically. Your password stays in the browser and is never entered into VS Code.

> ✅ **You'll know it worked when:** BugIt replies "✅ Activated — licensed to [your name/email]."

> **Keep your account private**
> Your account and its entitlement are tied to you. Don't share, post, or resell them — shared access can be revoked. Team members each sign in with their own account; there is no shared key and no shared login. Moving to a new computer later is fine (see FAQ).

6 Accept the terms (once)

The first time, BugIt shows a short summary and asks you to accept before it does anything else.

1. Read the short summary it shows (you review each ticket before it's filed; your project's safety is your responsibility; the safeguards are aids, not guarantees — and since they're followed by whichever AI model is answering you in Copilot, a lighter/free model may occasionally skip a step or need a command repeated, where a stronger model wouldn't).
2. Reply `I agree` and press Enter.

> ✅ **You'll know it worked when:** BugIt confirms and moves on to setup. You're only asked this once — it remembers.

7 Run "Begin setup"

Now BugIt configures itself by interviewing you in plain language. You never edit a settings file by hand.

```
YOU TYPE
Begin setup
```

It will ask short questions and turn on only what you pick. The most important one is your tracker:

It asks...	What to answer
Which tracker do you use? (pick one — required)	Where your team keeps bugs. **Jira and Azure DevOps** get guided setup with tested field mapping — severity/priority is set automatically to match your report. Jira uses the Atlassian Rovo MCP with browser sign-in; Azure DevOps uses Microsoft's organization-scoped remote MCP (a public preview), also via browser sign-in. **Sentry, GitHub, Linear, and Notion** use a known endpoint BugIt sets up and verifies live on your account — treat these as experimental until those checks pass. Everyone else (GitLab, Shortcut, YouTrack, Bugzilla, ClickUp, Asana, Trello) connects through your own compatible MCP server, which BugIt helps you set up. Pick the one you already use.
Crash tool? (optional)	Sentry uses a known endpoint that BugIt verifies live on your account (experimental); Crashlytics, BugSnag, Raygun, Rollbar, App Center, and Datadog connect via your own compatible MCP server — only if your team uses one.
Test management? (optional)	TestRail, Xray, Zephyr, Qase.
Post bugs to chat? (optional)	Slack, Teams, Discord, or email.
Specs / knowledge? (optional)	Confluence, Notion, SharePoint, Google Docs, or a local folder of Markdown files in this repo.
Languages	Your main language (default English). It asks a simple yes/no if you also file in a second language — most teams use just one.
Power features? (optional)	Extra tools like triage digest, undo, and offline export — see Part 2. All optional.

> **Tip: start from a template**
>
> If your work fits a category, type `preset.py game` (also `web-saas`, `mobile`, `enterprise`) to pre-fill sensible categories and settings. You can always change them later.

> **Setup pauses or gets interrupted? Just say it again**
>
> If `Begin setup` ever stops partway — you closed VS Code, or it just paused — typing `Begin setup` again picks up right where it left off. It won't make you re-answer questions you already answered. To deliberately change something later (add a tracker, swap a choice), just say so — e.g. "add a tracker" — and it reopens that part of the picker.

8 Connect your tracker (the "bridge")

Your tracker talks to BugIt through a small bridge (its technical name is an "MCP server"). BugIt sets this up **for you** — you just answer questions and click one button. You never edit files.

8a · Let BugIt configure the bridge

1. During setup, BugIt offers to connect your tracker. For Jira and Confluence (Atlassian Rovo) it knows the guided connection; for Sentry, GitHub, Linear, and Notion it knows the standard endpoint and verifies it live on your account — **confirm** when it asks, and it runs the checks before you rely on it (treat these as experimental until they pass).

2. For self-hosted or organization-supplied tools, it tells you in one sentence where to find your compatible MCP server address and asks you to **paste it into the chat** — then it writes it into the settings.

8b · Turn the bridge on

1. In the file list on the left, open the file `.vscode/mcp.json` (click it once).
2. Inside, look for small grey text that reads `▷ Start | More…` — it sits just above each server's name. It's small and easy to miss.
3. Click **Start** on each server BugIt told you to start (usually just your one tracker).

8c · Sign in when it asks

The first time a bridge connects, it needs to prove it's really you. One of two things happens:

- **A browser window opens** → sign in to your tracker as normal (common for GitHub and Azure DevOps).
- **A small box appears at the top of VS Code** → it asks for your email and/or an access token. Paste it and press Enter.

> **Where do I get an access token?**
> A token is like a one-time password your tracker gives you. Create one on your tracker's website, then paste it when the box asks. Here are the pages for the popular ones — sign in with the account you normally use:

Your tracker	Where to create a token
Jira / Confluence (Atlassian)	`id.atlassian.com/manage-profile/security/api-tokens`
GitHub	`github.com/settings/tokens` (or just sign in via the browser)
GitLab	`gitlab.com/-/user_settings/personal_access_tokens`
Sentry	sentry.io → Settings → Account → API → Auth Tokens
Linear	linear.app → Settings → API → Personal API keys
Azure DevOps	Usually signs in by browser — no token needed.

> **Copy your token the moment it's shown**
> Most services show a new token only once. Copy it immediately and keep it somewhere safe until it's stored in your computer's secure store (Step 9). Pick the longest expiry your tracker offers so you don't have to redo this soon.

8d · Check it's connected

After starting a bridge, its grey text changes to "Running" or shows a green dot. To be sure, type in the chat:

```
YOU TYPE
Check status
```

> ✅ **You'll know it worked when:** BugIt lists your tracker as connected / healthy.

The bridge turns off when you close VS Code

Each time you reopen VS Code, click **Start** on your server again in `.vscode/mcp.json`. Your saved sign-ins are remembered (Step 9) — you just restart the bridge. This is a VS Code behaviour, not a BugIt issue.

9 Save your sign-ins

Once you sign in to a bridge (Step 8c), your tracker's connector — or VS Code itself — remembers it in your computer's built-in secure store (Windows Credential Manager / macOS Keychain / Linux Secret Service), so you never have to look your token up again. BugIt keeps no copy of its own: `config.json` holds only orgs and URLs, and credential values never touch the chat log or a plain file.

To see which connectors are already signed in, type:

```
YOU TYPE
Check saved credentials
```

It reports which connectors own authentication — it **never shows credential values**. If a bridge later forgets your sign-in, just sign in again when it asks (Step 8c).

10 Confirm you're ready

Ask BugIt to double-check everything before you file for real:

```
YOU TYPE
Check readiness
```

It lists anything still missing (usually just "start your bridge" or "sign in"). When it's all green, you'll see "✅ Setup complete."

That's setup done — forever

You won't repeat Part 1. From now on, opening BugIt is: open the folder → start your bridge in `.vscode/mcp.json` → type `hi`. To change any choice later, just type `Begin setup` and say what to change.

No GitHub Copilot? There's a terminal wizard

If you can't use Copilot, open the built-in Terminal (Terminal → New Terminal) and run `python tools/setup_wizard.py`. It asks a few questions and writes your settings without the AI. You can also run BugIt standalone with your own AI key — see the FAQ.

PART 2 · EVERYDAY USE

Once set up, you work entirely from the chat. Talk naturally, or use the exact examples below. Type `help` anytime for the full list.

Write a bug report

BugIt's main job. Describe the bug in one plain sentence; it writes the full ticket, auto-fills the version and category, checks for duplicates, shows you a preview, and files it after your `yes`.

```
YOU TYPE
Write a bug report: the app crashes every time I tap Save on iPhone
```

It drafts a title, steps to reproduce, expected vs. actual result, severity, and platform — and it always sets the tracker's own priority/severity field to match, not just a generic default. Want changes before filing? Just say so — e.g. "make the severity high" or "add that it started in the latest build."

Quick bugs (fast filing)

For common shapes (crash, layout, slow, missing, blocker…), the quick form skips some questions and fills category and priority for you — and still runs the full duplicate check first.

```
YOU TYPE (EXAMPLES)
Quick bug: crash on startup after update
Quick bug: layout button overlaps text on the settings screen
Quick bug: blocker can't continue past the payment step
Quick bug: slow the dashboard takes 20 seconds to load
```

Crash bug reports

If you connected a crash tool (like Sentry), BugIt does everything above plus matches your report to the crash and pulls the stack trace for you.

```
YOU TYPE
Write a crash bug report: game crashes when opening the map
```

Verify & close comments

After you test a fix, BugIt writes a tidy comment to post on the ticket. It writes (and optionally posts) the comment — it does not change the ticket's status. You still close/resolve/reopen the ticket yourself in your tracker.

You type	What you get
`Close #123`	Asks which scenario (fix confirmed, closed per request, as-designed, reopen…) then writes that comment.
`Write a verification comment for 1.2.0 on Windows`	A "fix confirmed" comment with the build details filled in.
`Write a reopen comment for 1.2.0 on Windows`	An "issue still happens" comment (then you reopen the ticket yourself).

Translation

Translate reports or terms between your configured languages, using your team's exact wording (from your glossary — see Part 3).

```
YOU TYPE (EXAMPLES)
Translate to ja: the save button does nothing on the profile screen
Translate this bug report to English: [paste text]
```

Look things up (specs & glossary)

If you connected specs or filled your glossary, ask BugIt questions about your project.

```
YOU TYPE (EXAMPLES)
What is [your term]?
Walk me through the checkout flow
What's new in specs?
```

Duplicate detection

This happens automatically before every filing — BugIt searches your tracker for matching tickets (even ones worded a bit differently) and warns you, so you never file the same bug twice.

Power features (if you turned them on)

Type this	What it does
`Triage digest`	Instant standup: open bugs by area/severity, oldest flagged.
`Export bug to file`	Saves the report as Markdown/CSV/JSON instead of filing (great before your tracker is set up).
`Undo last filing`	Reverses the last ticket/comment where your tracker allows.
(automatic)	SLA flags on urgent bugs · category-specific fields.

The full command list

```
YOU TYPE
help
```

...shows every command inside the agent, anytime.

> **Practise safely anytime**
>
> Type `dry run` and BugIt writes the entire report **without** filing anything — perfect for learning or testing.

PART 3 · MAKE IT YOURS — ADD YOUR PROJECT'S KNOWLEDGE

Out of the box BugIt is a blank-slate QA agent. Here's how to teach it your project after purchase. No coding — mostly just talking to it in chat. Everything you add stays on your computer.

1 · Your team's words (the glossary)

This makes translations and category-guessing accurate for your project.

1. In the file list, open `.github/glossary/terms.template.md`.
2. Fill in the table with your team's words — feature names, screen names, item names, abbreviations, and (if you use two languages) their translations.
3. Save the file (`Ctrl+S`). That's it — BugIt uses these exact terms from now on.

> **Shortcut**
> Turn on "glossary auto-seed" during `Begin setup` and BugIt suggests terms from your existing tickets — you just approve each one.

2 · Your bug style (house style)

Want tickets to match your team's exact format — title style, severity labels, required fields? You don't describe it — you **show** it:

1. In the chat, say: "match my team's style."
2. Paste **one screenshot** of an existing ticket from your tracker.
3. BugIt studies it (locally, with any personal data stripped first) and saves your format, then follows it on every future ticket.

3 · Your categories, platforms & fields

Just tell BugIt in plain language and it updates your settings for you:

```
YOU TYPE (EXAMPLES)
My categories are Gameplay, UI, Audio, and Networking
We test on Windows and Xbox
```

4 · Your own tools (custom connections)

Use a tool that isn't in the built-in list? Connect it just like a built-in:

```
YOU TYPE
Add a custom MCP server
```

Give it a short name and its address (must be `https://`); BugIt connects and health-checks it for you.

5 · Just correct it as you go

The easiest way of all: when BugIt drafts a ticket, fix anything you don't like — a label, the wording, the severity. With "learn from your edits" turned on (recommended), it remembers your preference and applies it next time. Your project knowledge builds up naturally — and none of it ever leaves your machine.

> **Everything you add is yours and stays local**
> Your glossary, house style, categories, and learned preferences live on your computer, are backed up before every update, and are never sent to anyone.

6 · Sharing your setup with your team (Team license)

Once you've set up your glossary, house style, and tracker choices the way you want, give the rest of your team the exact same setup in one command instead of repeating steps 1–3 on every machine:

YOU RUN (ONCE, AFTER YOUR SETUP IS THE WAY YOU LIKE IT)

```
python tools/share.py export
```

That writes `config.share.zip` — your tracker/crash/comms choices, glossary, and house style, bundled together with no passwords or tokens inside. Send it to your team.

EACH TEAMMATE RUNS

```
python tools/share.py import config.share.zip
```

They get your exact terminology, bug style, and tracker setup immediately — no re-typing categories, no re-pasting a screenshot for house style. Their own license and device settings are untouched.

> **If the import changes where data is sent**
>
> If a shared setup adds a new custom connection or notification address, BugIt shows exactly what changed before anything else happens — it's not applied silently. It also snapshots the teammate's current setup first, so `Restore my settings` undoes the import if it's not what they expected.

PART 4 · UPDATES, BACKUPS & SAFETY

Getting updates (one command)

When a newer version is available, BugIt mentions it once when you say `hi`. Free v1.x updates are included while your license is active. To install it:

YOU TYPE

```
Update
```

1. BugIt takes a fresh backup of your files first.
2. It downloads the new version and checks it's genuine and untampered (it's cryptographically signed — a fake or altered update is refused).
3. It installs it without touching your settings, glossary, house style, license, or tokens.
4. It asks you to reload VS Code (`Ctrl+Shift+P` → type "Reload Window" → Enter). Start a fresh chat and you're on the new version.

> **No folder deleting, ever**
>
> Unlike the first install, updates are in-place. You never delete or re-download anything, and your setup is always preserved and backed up.

What's safe to hand-edit — and what isn't

Safe, and kept through every update: `config.json` , `.github/instructions/house-style.instructions.md` (see Part 3 — this is the supported way to customize behavior), `.github/glossary/terms.template.md` , `.github/instructions/learned.instructions.md` , and `.vscode/mcp.json` .

Not safe — don't hand-edit these

The agent file (`.github/agents/bugit-qa-agent.agent.md`), any other file under `.github/instructions/` , and anything in `tools/` are the product itself, not your settings. An update **silently overwrites** them, and — unlike the files above — **there is no backup to restore them from**. BugIt checks for this at startup and again right before installing an update, and warns you if it finds a change — but the safest rule is simply not to edit them.

Backups & restore

You type	What it does
`Back up my settings`	Saves a local snapshot of your config, glossary, house style, MCP endpoint settings, and learning. Your signed device entitlement and credential values are excluded.
`List backups`	Shows every saved snapshot with its date.
`Restore my settings`	Rolls back to a snapshot (and snapshots your current state first, so a restore is itself undoable).

Automatic before every update

BugIt always backs up before installing a new version, so an update can never lose your setup. If anything looks off, `Restore my settings` puts it right.

Safety — what's protected

✅ **Nothing without your yes**
Every ticket and comment is previewed; anything irreversible needs a direct Approve once in the bundled BugIt extension.

🧪 **Dry-run = practice mode**
Say `dry run` and BugIt's bundled Python helpers write nothing and the agent refuses tracker writes — so it only reads, never files, comments, or notifies. That refusal follows BugIt's instructions, not a platform/OS lock, so use read-only credentials when evaluating.

🔑 **No secrets in files**
Tokens live in your OS vault, never in a plain file, never sent to us.

🔒 **Runs on your machine**
It talks only to the tools you connect and your own AI model.

Your data & privacy

The only thing BugIt sends to Taskivator is license/update data: your BugIt account sign-in (in the browser, on the Portal), a one-way hashed device ID (a scrambled code that can't identify you), and the Solo or Team entitlement you approve for this device. There is no license key, and your password stays in the browser — it is never entered into VS Code. Your bug reports, specs, glossary, settings, and tokens never leave your machine. No telemetry, no analytics, ever. Full details are in the `PRIVACY.md` file in your BugIt folder.

Risks & important cautions — please read

AI can be wrong

BugIt drafts reports with an AI model, which can misread details or invent something. Always read the preview before you say "yes." You are approving what gets filed.

It files to your real tracker

Once you approve, a real ticket is created. Practising? Use `dry run` to write the report without filing.

Your project's safety is yours

How you use BugIt, and the safety of your projects, data, and trackers, are your responsibility. The safeguards reduce risk but don't replace your review.

What BugIt doesn't do

Tested field mapping (severity/priority set automatically) is Jira + Azure DevOps (Azure DevOps via Microsoft's remote MCP public preview); Confluence uses the same guided Atlassian connection; Sentry, GitHub, Linear, and Notion are experimental and verified live on your account; everything else connects via your own compatible MCP server (which you set up, with BugIt's help); it uses your AI (Copilot or your own OpenAI/Anthropic key), not a bundled model; redaction is best-effort; and it runs in VS Code only. One license = one device (Solo) or up to 5 members or device seats (Team). Results and consistency also depend on which AI model is answering inside Copilot — a stronger model follows the safety steps more reliably than a very lightweight/free one.

PART 5 · TROUBLESHOOTING & FAQ

Almost every bump is solved in seconds — and the fastest fix is usually to just ask the assistant.

✨ Try this first — ask the AI to fix it

Whenever anything goes wrong — during setup or everyday use — your quickest fix is almost always to **ask the assistant right in the chat**. Describe what happened in plain words (for example, "the setup stopped halfway" or "my tracker bridge won't connect") and ask it to fix it. The AI can diagnose and repair most problems on the spot. Please give this a try before emailing us — it resolves the large majority of issues in seconds.

Applying an AI fix: the "Keep" button

If the assistant fixes something by changing a file, VS Code shows a small Keep / Undo bar. If you asked for the fix and you're happy with it, click **Keep** — the change is applied to your own copy only, on your computer. Click **Undo** to discard it. Nothing you Keep is shared with anyone else.

What you see	What to do
"Activate to start"	Type `Activate`. BugIt opens the BugIt Portal in your browser — sign in, choose your Solo or Team entitlement, and approve this device. No account yet? Get one at the store.
The browser didn't open, or activation didn't finish	Type `Activate` again to reopen the BugIt Portal, then finish signing in and approving this device. Activation happens entirely in the browser — there is no key to paste.
"No tracker enabled"	Type `Begin setup` and pick your tracker.
Your bridge shows a red ✗ / won't connect	Open `.vscode/mcp.json` and click Start above the server. Still stuck? Type `fix servers` and follow the steps.
It keeps asking you to sign in	Say `fix servers`, restart the server, and complete the browser sign-in. BugIt never displays saved credential values. (First time? See Step 8c.)

It won't file a bug	Type `Check readiness` — it lists exactly what's missing (usually the bridge isn't started or you're not signed in).
Something looks broken after an update	Type `Restore my settings` to roll back.
The agent seems "off script"	Type `Read and follow all your given instructions`, or start a fresh chat. This can happen more often on a lightweight/free AI model — a stronger model in Copilot's model picker is more consistent.
Answers feel generic / not project-specific	Give it more context, or show it one existing ticket during `Begin setup` so it learns your style. Rerun `Begin setup` to refine.
You want to change a setup choice	Type `Begin setup` and say what to change (e.g. "add a tracker" or "start over") — it reopens just that. If setup was simply interrupted partway, a bare `Begin setup` resumes where it left off instead of restarting.

> **Built-in health checks**
> `Check status` (are my tools connected?) · `Check readiness` (what's left before I can file?). For a deeper check, open Terminal → New Terminal and run `python tools/selftest.py`.

Frequently asked questions

Do I need to know how to code?
No. If you can install an app and type a sentence, you can use BugIt. It does the technical parts for you.

Will it file bugs without me knowing?
Never. Every ticket and comment is previewed first, and an irreversible action needs a direct `Approve once` click in the bundled BugIt extension — typing `FILE` or `COMMENT` in chat only signals intent, and chat text alone never authorizes it. To practise, say `dry run`: BugIt's bundled Python helpers write nothing and the agent refuses tracker writes, though that refusal follows BugIt's instructions rather than a platform/OS lock, so use read-only credentials when evaluating.

Do I have to start the bridge every time?
Yes — open `.vscode/mcp.json` and click Start when you reopen VS Code. Your saved sign-ins are remembered; only the bridge needs restarting.

Can I use it on two computers?
One device at a time on Solo (a Team covers up to 5 members or device seats, one per member). Moving to a new device is fine: activate the new one, and the old device stops being able to renew straight away. Its slot frees up as soon as the old device reconnects (confirming it has given up access) or its current 72-hour offline lease runs out, so there is no fixed waiting period. Type `License status` to check.

What if the licensing server is down?
You keep working. After a successful online activation, BugIt runs on a cached lease for up to 72 hours offline, so a server outage (or just a weekend with no internet) won't interrupt you within that window. After that it re-verifies online.

Do I need GitHub Copilot?
It's the easiest way. If you don't have it, run `python tools/setup_wizard.py` in the Terminal to set up, and BugIt can run standalone with your own OpenAI/Anthropic key (`python standalone/run.py`).

Is my data private?
Yes. The only thing BugIt sends to Taskivator is license/update data — your BugIt account sign-in (in the browser, on the Portal), a hashed device ID, and the Solo or Team entitlement you approve for this device — no telemetry, ever. Your tickets themselves go only to the AI model and tracker you connect, never to us.

Can I edit BugIt's files?
You're meant to customize your parts — your glossary, house style, and settings (Part 3). Just don't disable the licensing/activation. If you hit a real bug in the tool itself, email support so it can be fixed for everyone in the next update.

Which bug trackers does it work with?
Jira and Azure DevOps get guided setup with tested field mapping (severity/priority set automatically) — Jira through the Atlassian Rovo MCP, Azure DevOps through Microsoft's remote MCP public preview. Sentry, GitHub, Linear, and Notion use a known endpoint that BugIt sets up and verifies live on your account (experimental until those checks pass); GitLab, Shortcut, YouTrack, Bugzilla, ClickUp, Asana, and Trello connect through your own compatible MCP server — BugIt writes the ticket and files it through whichever connector you enable. Pick yours during setup and switch anytime with `Begin setup`.

Will it catch duplicate bugs before I file?
Yes. Before anything is written, it searches your tracker for similar reports — even ones worded a little differently — and shows you the matches, so you can avoid raising the same bug twice. You still decide whether to go ahead.

Can it write tickets in more than one language?
Yes. Add a second language during setup and every report is written in both, kept in separate blocks, using your glossary so product terms stay exact — it never improvises a translation.

Can I attach a screenshot?
Just paste the image into the chat. BugIt reads it, scrubs anything sensitive it notices, and attaches it to the ticket once you approve.

Does it hide passwords and tokens in my reports?
When redaction is switched on, it scrubs emails, tokens, and IP addresses out of every draft before filing. You always see the full text in your preview first.

I filed a ticket by mistake — can I undo it?
The preview and typed "yes" stop most mistakes before anything is written. If you turned on the audit log, type `Undo last filing`. Otherwise, just close or edit the ticket in your tracker as usual.

Can it help me verify or close a bug?
Yes. Ask for a verification comment and it drafts a clear pass/fail note (in your languages) on the existing ticket — you approve it before it posts.

Can I use it for more than one project?
Each BugIt folder is set up for one project's tracker. For another project, either re-run `Begin setup` to point it somewhere new, or keep a separate copy per project.

How do I get updates?
When a new version is out, BugIt mentions it as you start a chat. Type `update` and it installs in place — your glossary, house style, and settings are kept, and it backs them up first.

What happens when my license expires?
When your term ends, renew from your account; your device applies the renewed entitlement the next time it checks in online. The 72-hour offline lease above is separate — it only covers a temporary licensing-*server* outage, not an expired term. Type `License status` anytime to see where you stand.

If I renew or buy another license, do the days stack?
No — licenses don't stack. A renewal **replaces** your current entitlement; its term starts fresh from the day it takes effect, and any unused days don't carry over. So renew near the end of your current term, not early.

PART 6 · LICENSE & SUPPORT

License terms (plain-language summary)

The full, binding terms are in the `LICENSE` file in your BugIt folder — that document governs. In short:

Topic	In plain words
Licensed, not sold	You buy a license to use BugIt, not ownership. Taskivator keeps all rights.
Your seats	Solo = one device at a time; Team = up to 5 members or device seats. Your account and entitlement are personal to you — don't share, resell, or publish them.
Revocation	Shared, refunded, charged-back, or misused access can be revoked.
Customize, but…	Edit your config, glossary, and templates freely — but don't disable or bypass the licensing/activation.
Your responsibility	How you use BugIt and your project's safety are yours. You review and approve every report before it's filed. The safeguards are aids, not guarantees.
"As is", liability	Provided as-is, no warranty. To the extent the law allows, liability is capped at what you paid, excluding indirect/consequential damages.
Term & law	A one-year license starting the day you activate — a one-time purchase, no auto-renew — governed by the laws of Japan. Refunds are handled through the store you bought from.
Renewals don't stack	A renewal replaces your current entitlement — its term starts fresh and unused days don't carry over, so renew near the end of your term. When a term ends, renew from your account and your device re-authorizes on its next online check.

> **Two documents in your folder**
> `LICENSE` (full terms) and `PRIVACY.md` (privacy statement). By activating and using BugIt, you accept the LICENSE.

Getting help

Please check Troubleshooting and the FAQ above first — they cover almost everything. Then, in this order:

1 · Ask the AI to fix it ⭐
Your best first move: describe the problem in the chat and ask the assistant to fix it. If it changes a file and it looks right, click Keep to apply it (your PC only).

2 · Run a health check
`Check status` · `Check readiness` · or `python tools/selftest.py` in the Terminal.

3 · Restore
`Restore my settings` undoes a bad change or update.

4 · Reach a human
Only if the guide and the AI couldn't help: visit bugit.dev or email support@bugit.dev (support is handled in English). Setup trouble in your first 7 days? We'll get you running or help with a store refund.

Please keep confidential project information out of support emails

Every project is different, and much of your work may be confidential. If you do email us, please describe the BugIt problem in general terms only — don't paste confidential code, bug details, specifications, screenshots, or data from your project. Taskivator is not responsible for any confidential information sent to us, and anything confidential we receive is deleted immediately. Wherever possible, ask the AI assistant to help first — it can resolve most issues right inside your editor.

Thank you for choosing BugIt.
File clean tickets, skip the duplicates, and get your time back — one bug at a time.